

FaithBloom Book Studio

Relatório Consolidado — Arquitetura, Histórico de Melhorias e Roadmap

Cobre todo o trabalho realizado em conjunto com o assistente de IA entre 16/09/2026 e 03/10/2026: auditoria de código real, correções de bugs, extensão do roteamento de IA (texto, voz e imagem) para o Gemini (Google AI Studio), implantação do Engenheiro de Code Review com execução real, e o que ainda falta para o SaaS funcionar plenamente.

Preparado para Erica Matsuzaki — 3 de outubro de 2026

1. Sumário executivo

O FaithBloom Book Studio é um SaaS editorial com IA para livros infantis cristãos originais, com a missão de "transformar histórias em sementes de fé". O projeto já possui uma base editorial ampla — 23 agentes especialistas com contratos de skills, Character Universe, Heart Arc, Prompt-Mestre, Quality Guardian — construída ao longo de meses em uma única feature branch (428 commits, 194 arquivos alterados) ainda em Draft, sem merge na release protegida.

O trabalho desta fase, feito em conjunto com o assistente, teve dois eixos: (1) **auditoria cruzada** entre o que os dossiês/conversas anteriores descreviam e o que o código realmente fazia — revelando divergências reais (descrição do PR à frente do código, roteamento de IA que não existia de fato); e (2) **implementação e correção real** de três capacidades: o SaaS deixar de depender só da OpenRouter (sem crédito) e passar a usar o Gemini gratuito/econômico para texto, voz e imagem; um bug real de travamento em interrupções do Streamlit; e um Engenheiro de Code Review Profundo que lê código de verdade, propõe correção via IA e abre Pull Requests reais para revisão — nunca mesclando sozinho.

Estado no fechamento deste relatório: CI verde (pytest + fullstack-audit) no PR #18, Gemini ativo para as três modalidades de IA, Engenheiro operacional com GITHUB_TOKEN configurado. Os itens pendentes mais importantes são a correção da descrição do PR (ainda descreve um estado do código que não existe), a ponte editorial do livro da Mel, e a extensão do mesmo padrão de confiabilidade para os livros de colorir — tema do próximo passo combinado com a autora.

2. Linha do tempo das mudanças reais (16/09 a 03/10/2026)

Data	O que foi feito	Resultado
16-17/09	gemini_client.py criado; openrouter_client.chamar_llm passa a tentar Gemini antes da OpenRouter	CI verde; Jarvis responde em texto sem crédito na OpenRouter
17/09	Fallback Gemini estendido para voz (TTS) do Jarvis	CI verde; voz ainda falhava por um bug separado
19/09	Bug corrigido: Jarvis sempre passava um `model` explícito para gerar_audio(), o que pulava o Gemini	Voz do Jarvis confirmada funcionando de verdade no app publicado
23/09	Engenheiro de Code Review Profundo implementado (engenheiro_code_review.py, tela manual, testes)	Pode investigar bugs reais e abrir PR Draft; depende de GITHUB_TOKEN
24/09	Gatilho automático: Jarvis aciona o Engenheiro sozinho quando a voz falha de verdade	Arquivo do Jarvis foi apagado e recriado pela autora com emoji diferente, quebrando navegação e CI — restaurado copiando o nome exato
28/09	GITHUB_TOKEN configurado nos Secrets do Streamlit	Engenheiro passa a conseguir abrir PR de verdade, não só diagnosticar
02/10	Fallback Gemini estendido para IMAGEM (gerar_imagem)	Texto, voz e imagem agora tentam o Gemini antes da OpenRouter; CI verde

3. Arquitetura macro atual

Camada	Função	Estado real em 03/10/2026
Jarvis	Entrada conversacional/multimodal, voz, calendário, roteador para especialistas	Texto, voz e handoff multimodal funcionando; Engenheiro acionado automaticamente em falha real de voz
Roteamento de IA (gemini_client.py + openrouter_client.py)	Decide o provedor de texto/voz/imagem	Gemini (gratuito/econômico) primeiro, OpenRouter como reserva, nas 3 modalidades
Engenheiro de Code Review Profundo	Investiga bug relatado, lê código real, propõe correção, abre PR	Operacional (tela manual + gatilho automático do Jarvis); nunca mescla sozinho
Orquestrador / graph.py	Coordena criação de livro novo: tema → roteirista → revisor (loop) → ilustrador → publicação	Implementado e testado para livro novo
Book Doctor	Diagnóstico de obra publicada, preserva original, lista pendências	Só diagnóstico; a ponte para a cadeia editorial (Revisor/Editor/Roteirista) ainda não existe
Character Universe	Character DNA, Color Master, Reference Pack, histórico	Forte; Mel madura (DNA visual canônico), Manu validada, Téio em conclusão
Compliance / Editorial DNA	Heart Arc, moral obrigatória, versículo protegido, psicologia das cores	Presentes nos contratos de skill e regras do Roteirista/Editor
Cost & Security	Modos de custo, consentimento antes de gasto, logs sanitizados, dedup de cliques	Implementado (controle_geracao.py); bug de travamento em interrupção corrigido
Persistência	Local + Supabase (cloud)	Integração feita; validação ponta a ponta pós-redeploy ainda pendente
Coloring / Activity Books	Geração de livros de colorir e atividades	Próximo item a ser auditado junto com a autora

4. Roteamento de IA (texto, voz e imagem) — detalhe técnico

O núcleo da mudança desta fase foi criar **gemini_client.py**, um cliente direto para a API gratuita/econômica do Google AI Studio, e transformar **openrouter_client.py** em um despachante: cada função pública (chamar_llm, gerar_audio, gerar_imagem) tenta o Gemini primeiro quando ele está configurado e disponível, e só cai para a OpenRouter paga se o Gemini falhar ou não estiver configurado. Nenhum agente precisou mudar seu código — a interface pública continua a mesma.

Regras de segurança mantidas na extensão:

- Quando quem chama já pede explicitamente um provedor/modelo específico da OpenRouter (ex.: Audiobook Studio escolhendo um modelo, ou Restoration Studio pedindo roteamento *google-vertex*), essa escolha é respeitada e o Gemini direto não é tentado por cima.
- Um bug real já existente no código (não causado por esta mudança) foi corrigido: quando o Streamlit interrompe uma geração no meio (usuário navega/atualiza a página), o sistema ficava com um travamento fantasma, porque capturava apenas *Exception* e não *BaseException*. Corrigido em texto, voz e imagem.
- Toda chamada ao Gemini continua passando pelo mesmo módulo de custo/segurança/duplicidade (controle_geracao.py) já usado pela OpenRouter — sem lógica de orçamento duplicada.

5. Agentes especialistas e skills (herdado da auditoria anterior, ainda válido)

Levantamento feito diretamente de agent_skills.py e skills/agent_profiles.json (schema faithbloom.agent-skills.v1, 23 papéis reais — a descrição do PR #18 ainda menciona "v3, 27 papéis", divergência que segue pendente de correção).

Agente	Situação
Gerador de Ideias, Curador de Tema	Implantados e acionados
Roteirista, Editor de História, Revisor Editorial	Implantados e acionados; loop Roteirista↔Revisor real para livro novo
Criador de Personagem, Foto→Personagem, Variações	Implantados e acionados
Ilustrador	Implantado e acionado
Ideias/Atividades/Line Art para Colorir	Implantados; auditoria mais profunda pendente (próximo passo combinado)
Diagramador (miolo e colorir)	Implantado, roda por regra determinística, sem IA
Especialista de Capa, Dedicatória, Sinopse, Marketing, Keywords/Categorias	Implantados e acionados
Tradutor & Localizador	Implantado e acionado
Diretor de Audiobook, Narrador/TTS	Implantados; execução de voz agora também usa Gemini
Editor Pedagógico	Fora do cadastro central; recebia cenas resumidas (não completas) — pendente de reforço
Diretor de Cena (Scene Director)	Fora do cadastro central; faltava faixa etária/contexto completo — pendente de reforço
Engenheiro de Code Review Profundo	Agora com execução real: lê código, propõe correção, abre PR Draft
Engenheiro Full Stack / Automação	Ainda só gera diagnóstico/plano; execução real não implementada

6. Engenheiro de Code Review Profundo — execução real

Antes desta fase, o "Engenheiro" era só um papel conversacional que descrevia um diagnóstico em texto. Agora existe um módulo real (engenheiro_code_review.py) que:

- Lê o conteúdo real do arquivo suspeito diretamente do GitHub (API oficial, com GITHUB_TOKEN próprio — diferente do acesso do assistente de IA usado nesta conversa);
- Usa o modelo de texto já configurado (Gemini/OpenRouter) para diagnosticar a causa raiz e propor uma correção completa do arquivo;
- Só age se a confiança reportada for "alta" — caso contrário, não mexe em nada e explica o motivo;
- Cria uma branch isolada (nunca a feature branch nem a release), commita, e abre um Pull Request em modo Draft;
- Nunca mescla sozinho — toda integração depende de revisão humana e do CI.

Acionamento: manual, pela tela "■■■ Engenheiro" no app, ou automático, quando o Jarvis encontra uma falha técnica real (hoje: falha de geração de voz) — com deduplicação por sessão para não abrir PRs repetidos pela mesma falha intermitente.

7. O que falta implantar — roadmap priorizado

Corrigir a descrição do PR #18 (Baixo risco, alto valor de clareza)

A descrição ainda afirma "Cadastro v3 com 27 papéis"; o código real tem 23 papéis em schema v1. Isso confunde qualquer decisão futura baseada só na leitura do PR.

Ponte editorial do livro da Mel (P0 do produto)

Ligar o Book Doctor à cadeia Revisor → Editor de História → Roteirista → motores compartilhados → Restoration Studio → Quality Guardian, reaproveitando os agentes já existentes — ainda não existe.

Auditoria dos livros de colorir (Próximo passo combinado com a autora)

Levantar o que já funciona (Ideias/Atividades/Line Art para Colorir) e o que falta para gerar um livro de colorir completo de ponta a ponta com a mesma confiabilidade alcançada para o livro ilustrado.

Fortalecer Editor Pedagógico e Diretor de Cena (Pré-requisito para mais automação)

Formalizar os dois no cadastro central de skills, com contexto completo (cenas completas, faixa etária, DNA dos personagens) em vez de dados resumidos.

Execução real do Engenheiro Full Stack (Mais delicado, fazer com cautela)

Hoje só gera plano/diagnóstico. Dar a ele o mesmo tipo de execução real do Code Review Profundo exigiria a mesma trava de segurança (PR Draft, nunca merge automático).

Validação de persistência Supabase ponta a ponta (Antes de confiar dados de clientes reais)

Confirmar que os dados sobrevivem a reboot/redeploy de forma comprovada, não só teoricamente integrada.

Testar o Engenheiro com um bug real (Validação final da Fase 4)

Já tem GITHUB_TOKEN configurado; falta um teste de ponta a ponta com um bug real para confirmar que o PR sai exatamente como desenhado.

8. Regras que continuam valendo em qualquer próximo passo

- Nunca mexer ou mesclar na release protegida (release/2.0.0-rc5-skills) sem validação real de runtime.
- Nenhum Master de personagem é promovido automaticamente; aprovação humana explícita sempre.
- Verificar → reutilizar → estender → criar somente se realmente não existir.
- CI verde é necessário, mas não suficiente — validação real no Streamlit publicado continua obrigatória.
- Duas IAs (este assistente e o ChatGPT Plus) não devem editar a mesma branch ao mesmo tempo — já causou um conflito real nesta mesma fase.